



Brief paper

On the computation of linear model predictive control laws[☆]Francesco Borrelli^{a,*}, Mato Baotic^b, Jaroslav Pekar^c, Greg Stewart^d^a Department of Mechanical Engineering, University of California, Berkeley, 94720-1740, USA^b Faculty of Electrical Engineering and Computing, University of Zagreb, Unska 3, 10000 Zagreb, Croatia^c Honeywell Prague Laboratory, V Parku 2326/18, 14800 Prague 4, Czech Republic^d Honeywell Automation and Control Solutions, North Vancouver, BC V7J 3S4, Canada

ARTICLE INFO

Article history:

Received 10 June 2009

Received in revised form

22 December 2009

Accepted 21 February 2010

Available online 8 April 2010

Keywords:

Model predictive control

Explicit MPC

Active set QP solver

Fast MPC

ABSTRACT

Finite-time optimal control problems with quadratic performance index for linear systems with linear constraints can be transformed into Quadratic Programs (QPs). Model Predictive Control requires the on-line solution of such QPs. This can be obtained by using a QP solver or evaluating the associated explicit solution. The objective of this note is twofold. First, we shed some light on the computational complexity and storage demand of the two approaches when an active set QP solver is used. Second, we show the existence of alternative algorithms with a different tradeoff between memory and computational time. In particular, we present an algorithm which, for a certain class of systems, outperforms standard explicit solvers both in terms of memory and worst case computational time.

© 2010 Elsevier Ltd. All rights reserved.

1. Introduction

In Bemporad, Morari, Dua, and Pistikopoulos (2002) the authors have shown how to compute the solution to constrained finite-time optimal control (CFTOC) problem for discrete-time linear systems as a piecewise affine (PWA) state-feedback law. Such a law is computed off-line by using a multi-parametric programming solver which divides the state space into polyhedral regions, and for each region determines the linear gain and offset which produces the optimal control action. This state-feedback law is often referred to as the “explicit solution”.

At each sampling time a Model Predictive Control (MPC) strategy (Mayne, Rawlings, Rao, & Scokaert, 2000) requires the solution of an open-loop CFTOC problem, which, for a quadratic performance index and known (measured/estimated) system state, corresponds to solving a Quadratic Program (QP). Having a precomputed solution as an explicit piecewise affine function of the state vector reduces the on-line computation of the MPC control law to a function evaluation, thus avoiding the on-line solution of a quadratic program.

The objective of this note is twofold. The first part sheds some light on the complexity of the on-line solution of a quadratic program (by means of an active set QP solver) versus the on-line evaluation of the explicit solution (by means of an “explicit solver”). We focus on the three main components of an active set QP algorithm and of an explicit solver: (1) the amount of stored data, (2) the optimality certificate, i.e. validation of the optimality conditions and (3) the selection of the next active set if the validation of the optimality fails. We carefully compare the three components in active set QP algorithms and explicit solvers and pinpoint the tradeoff between computational time and memory requirements. The second part builds on the first part and shows that there is a range of alternative approaches which trade off memory and computational time in a different way compared to both active set QPs and explicit solvers. In particular we present an algorithm that, for certain classes of systems, outperforms explicit solvers both in terms of memory and computational time.

We remark that the main objective of this paper is to show that, depending on the problem and on the hardware platform, there might be alternative and more efficient ways for computing explicit linear MPC laws. However, the corresponding state-feedback controllers do not have the nice piecewise affine closed-form solution as the explicit solution presented in Bemporad et al. (2002). It is not the intent of this paper to compare the proposed algorithms with other very efficient explicit solvers appeared in the literature (Baotic, Borrelli, Bemporad, & Morari, 2008; Jones, Grieder, & Raković, 2005), or fast QP solvers (Ferreau, Bock, & Diehl, 2008; Milman & Davison, 2008; Santos, Afonso, Castro, Oliveira, & Biegler, 2001; Wang & Boyd, 2008) tailored to the

[☆] The material in this paper was partially presented at European Control Conference August 2009, Budapest and the Conference on Decision and Control, December 2009, Shanghai. This paper was recommended for publication in revised form by Associate Editor Martin Guay under the direction of Editor Frank Allgöwer.

* Corresponding author. Tel.: +1 510 643 3871; fax: +1 510 643 5599.

E-mail addresses: fborrelli@me.berkeley.edu (F. Borrelli), mato.baotic@fer.hr (M. Baotic), jaroslav.pekar@Honeywell.com (J. Pekar), greg.stewart@Honeywell.com (G. Stewart).

specific structure of the underlying optimal control problem or suboptimal explicit solution. The comparison would be problem dependent and requires the simultaneous analysis of several issues such as speed of computation, storage demand and real time code verifiability. This is an involved study and as such is outside of the scope of this paper.

2. Notation

Throughout this paper (lower and upper case) italic letters denote scalars, vectors and matrices (e.g., $A, a, \dots$), while upper case calligraphic letters denote sets (e.g., $\mathcal{A}, \mathcal{B}, \dots$). $\mathbb{R}$ is the set of real numbers, $\mathbb{N}$ is the set of positive integer numbers. For a matrix (vector) A , A' denotes its transpose, while A_i denotes the i th row (element). For a set $\mathcal{A}$, $|\mathcal{A}|$ denotes its cardinality (number of elements), and $\mathcal{A}(k)$ denotes the k th element. Given the matrix $G \in \mathbb{R}^{m \times n}$, then for any set $\mathcal{A} \subseteq \{1, \dots, m\}$, $G_{\mathcal{A}}$ denotes the submatrix of G consisting of the rows indexed by $\mathcal{A}$. $Q > 0$ denotes positive definiteness (resp., $Q \geq 0$ positive semidefiniteness) of a square matrix Q . With $\ell := \rho$ ($\ell := \rho$) we denote that ℓ is defined by ρ (ρ is defined by ℓ).

3. CFTOC and its state-feedback PWA solution

Consider the discrete-time linear time-invariant system

$$x(t+1) = Ax(t) + Bu(t) \quad (1)$$

subject to the constraints

$$E^x x(t) + E^u u(t) \leq E \quad (2)$$

at all time instants $t \geq 0$.

In (1)–(2), $n_x \in \mathbb{N}$, $n_u \in \mathbb{N}$ and $n_E \in \mathbb{N}$ are the number of states, inputs and constraints respectively, $x(t) \in \mathbb{R}^{n_x}$ is the state vector, $u(t) \in \mathbb{R}^{n_u}$ is the input vector, $A \in \mathbb{R}^{n_x \times n_x}$, $B \in \mathbb{R}^{n_x \times n_u}$, $E^x \in \mathbb{R}^{n_E \times n_x}$, $E^u \in \mathbb{R}^{n_E \times n_u}$, $E \in \mathbb{R}^{n_E}$, and the vector inequality (2) is considered elementwise.

Let $x_0 = x(0)$ be the initial state and consider the constrained finite-time optimal control (CFTOC) problem

$$\begin{aligned} J^*(x_0) &:= \min_U \quad x_N' Q^{xN} x_N + \sum_{k=0}^{N-1} x_k' Q^x x_k + u_k' Q^u u_k \\ \text{subj. to} \quad &\begin{cases} x_{k+1} = Ax_k + Bu_k, \\ E^x x_k + E^u u_k \leq E, \\ k = 0, \dots, N-1 \end{cases} \end{aligned} \quad (3)$$

where $N \in \mathbb{N}$ is the horizon length, $Q^x = (Q^x)' \geq 0$, $Q^u = (Q^u)' > 0$, $Q^{xN} = (Q^{xN})' \geq 0$, $U := [u_0', \dots, u_{N-1}']' \in \mathbb{R}^{n_u N}$ is the optimization vector, x_i denotes the state at time i if the initial state is x_0 and the control sequence $\{u_0, \dots, u_{i-1}\}$ is applied to the system (1), $J^* : \mathbb{R}^{n_x} \rightarrow \mathbb{R}$ is the value function.

Consider the problem of regulating to the origin the discrete-time linear time-invariant system (1) while fulfilling the constraints (2). The solution U^* to CFTOC problem (3) is an open-loop optimal control trajectory over a finite horizon. A Model Predictive Control (MPC) (Mayne et al., 2000) strategy employs it to obtain a feedback control law in the following way: assume that a full measurement of the state $x(t)$ is available at the current time $t \geq 0$. Then, the CFTOC problem (3) is solved at each time t for $x_0 = x(t)$, and

$$u(t) = u_0^* \quad (4)$$

is applied as an input to system (1).

3.1. Solution of CFTOC

Consider the CFTOC problem (3). This can be rewritten as the quadratic program (Bemporad et al., 2002)

$$\begin{aligned} J^*(x) &= \min_U \quad \frac{1}{2} U' H U + x' F U + \frac{1}{2} x' W x \\ \text{subj. to} \quad &G U \leq b^r + B^x x \end{aligned} \quad (5)$$

where $x = x_0$, the column vector $U := [u_0', \dots, u_{N-1}']' \in \mathbb{R}^n$, $n := n_u N$, is the optimization vector, $H = H' > 0$, and H, F, W, G, B^x, b^r are easily obtained from Q^x, Q^u, Q^{xN} and (3) (see Bemporad et al. (2002) for details). Because the problem depends on x the implementation of MPC can be performed either by solving the QP (5) on-line or, as shown in Bemporad et al. (2002), by solving problem (5) off-line for all x within a given range of values, i.e., by considering (5) as a multi-parametric Quadratic Program (mp-QP).

In Bemporad et al. (2002) the authors give a self-contained proof of the following properties of the mp-QP solution.

Theorem 1. Consider the multi-parametric quadratic program (5) and let $H > 0$. Then the set of feasible parameters $\mathcal{X}_f \subseteq \mathbb{R}^{n_x}$ is convex, the optimizer $U^* : \mathcal{X}_f \rightarrow \mathbb{R}^n$ is continuous and piecewise affine (PWA), and the value function $J^* : \mathcal{X}_f \rightarrow \mathbb{R}$ is continuous, convex and piecewise quadratic.

Once the multi-parametric problem (5) is solved off-line, i.e., the solution $U^*(x) = f_{\text{PWA}}^*(x)$ of the CFTOC problem (5) is found, then the state-feedback PWA MPC law can be simply obtained by extracting first n_u elements of $f_{\text{PWA}}^*(x)$

$$u^*(t) = [I_{n_u} \mathbf{0}_{n_u \times (n-n_u)}] f_{\text{PWA}}^*(x(t)). \quad (6)$$

Therefore by using a multi-parametric solver the computation of a MPC law becomes a piecewise affine function evaluation.

4. Active set algorithms versus explicit solvers

The objective of this section is to compare computational time and storage demand associated to (i) active set QPs for solving (5) and to (ii) the evaluation of the explicit solution (6). For the sake of better readability we rewrite (5) compactly as

$$\begin{aligned} \min_U \quad &\frac{1}{2} U' H U + g(x)' U \\ \text{subj. to} \quad &G U \leq b(x) \end{aligned} \quad (7)$$

where $G \in \mathbb{R}^{m \times n}$, $b(x) \in \mathbb{R}^m$, $g(x) \in \mathbb{R}^n$, $g(x) := F'x$ and $b(x) := b^r + B^x x$. Here we assume, without loss of generality, that $W = 0$. The solution to the problem (7) for a fixed x is fully characterized by the set of Karush Kuhn Tucker (KKT) conditions:

$$H U^* + G_{\mathcal{A}^*}' \lambda^* + g(x) = 0 \quad (8a)$$

$$G_{\mathcal{A}^*} U^* = b_{\mathcal{A}^*}(x), \quad (8b)$$

$$\lambda^* \geq 0, \quad (8c)$$

$$G_{\mathcal{I} \setminus \mathcal{A}^*} U^* < b_{\mathcal{I} \setminus \mathcal{A}^*}(x), \quad (8d)$$

where $\mathcal{I}$ denotes the set of all constraint indices $\mathcal{I} := \{1, \dots, m\}$, $\mathcal{A}^* = \mathcal{A}^*(x)$ is the set of active constraints at $U^*(x)$:

$$\mathcal{A}^*(x) := \{j \in \mathcal{I} : G_j U^*(x) = b_j(x)\}, \quad (9)$$

and $\lambda^* = \lambda^*(x) \in \mathbb{R}^{|\mathcal{A}^*|}$ are Lagrange multipliers.

In the optimization field, the variety of QP algorithms is very rich and their performance depends on the type of problem. In this note, we prefer to highlight the main differences between the two approaches (on-line versus explicit) and corresponding changes in computational time and storage demand rather than selecting a specific QP implementation and carrying an exact computation. For this reason, in the following Section 4.1, we present the main steps of a simple active set QP algorithm and in Section 4.2 we present the main steps of an explicit solver. For the same reason, we will not cover the variety of pivoting rules for the degenerate QP case. If the presented algorithms terminate without assigning U^* , then the problem (7) is infeasible for the given x .

4.1. Active set QP solver

Before presenting an active set method for solving the QP (7), we will first consider a subset $\mathcal{A}$ of the constraints index $\mathcal{A} \subseteq \mathcal{I}$

and the following equality constrained QP:

$$\begin{aligned} \min_U \quad & \frac{1}{2} U' H U + g(x)' U \\ \text{subj. to} \quad & G_{\mathcal{A}} U = b_{\mathcal{A}}(x). \end{aligned} \quad (10)$$

A Lagrangian method (Fletcher, 1987, p. 229) solves the equality constrained QP (10) by computing a solution to the KKT conditions, compactly written as:

$$\begin{bmatrix} H & G'_{\mathcal{A}} \\ G_{\mathcal{A}} & 0 \end{bmatrix} \begin{bmatrix} U^* \\ \lambda^* \end{bmatrix} = \begin{bmatrix} -g(x) \\ b_{\mathcal{A}}(x) \end{bmatrix}. \quad (11)$$

The solution to (11) has the form

$$U^* = -Lg(x) + Tb_{\mathcal{A}}(x), \quad (12a)$$

$$\lambda^* = -T'g(x) + Sb_{\mathcal{A}}(x). \quad (12b)$$

The symmetric matrix in (11) is referred as the Lagrangian matrix. If the Lagrangian matrix is nonsingular then the matrices L , T and S can be extracted from its inverse

$$\begin{bmatrix} L & T \\ T' & S \end{bmatrix} = \begin{bmatrix} H & G'_{\mathcal{A}} \\ G_{\mathcal{A}} & 0 \end{bmatrix}^{-1}. \quad (13)$$

Several alternatives for solving (13) can be found in Fletcher (1987, p. 236). In particular, there are instances when the Lagrangian matrix in (11) is not invertible. This happens when $G_{\mathcal{A}}$ is not a full row-rank matrix. In this case any feasible point U can be written as $U = Yb_{\mathcal{A}}(x) + Zy$, where $y \in \mathbb{R}^{n-m_1}$, with m_1 being the rank of $G_{\mathcal{A}}$. One possibility for computing U^* and λ^* is to use the QR factorization of the matrix $G_{\mathcal{A}}$

$$G'_{\mathcal{A}} = Q \begin{bmatrix} R \\ 0 \end{bmatrix} = [Q_1 \ Q_2] \begin{bmatrix} R \\ 0 \end{bmatrix} = Q_1 R \quad (14)$$

where $Q \in \mathbb{R}^{n \times n}$ is orthogonal, $R \in \mathbb{R}^{m_1 \times m_1}$ is upper triangular and $Q_1 \in \mathbb{R}^{n \times m_1}$ and $Q_2 \in \mathbb{R}^{n \times (n-m_1)}$. Then with the choice of

$$Y = Q_1 R^{-1}, \quad Z = Q_2, \quad (15)$$

we get (cf. Fletcher, 1987, Eq. 10.2.7) the solution (12) with

$$\begin{aligned} L &= Z(Z'HZ)^{-1}Z', \\ T &= -Y + Z(Z'HZ)^{-1}Z'HY, \\ S &= -Y'HY + Y'HZ(Z'HZ)^{-1}Z'HY. \end{aligned} \quad (16)$$

Alternative approaches for computing U^* and λ^* can be found in Fletcher (1987). We remark that they all consists in manipulating (11) and, after a certain number of operations, obtaining U^* and λ^* .

A primal active set method is one of the most common method for solving the QP (7) for a fixed x . Next the main ingredients of its implementation extracted from Fletcher (1987) are briefly recalled.

At each iteration k the following auxiliary problem is solved

$$\begin{aligned} \min_{\delta} \quad & \frac{1}{2} \delta' H \delta + (g(x) + HU)' \delta \\ \text{subj. to} \quad & G_{\mathcal{A}} \delta = 0 \end{aligned} \quad (17)$$

where $U = U^{(k)}$ is a known feasible point for the QP (7) and $\mathcal{A} = \mathcal{A}^{(k)} := \{j \in \mathcal{I} : G_j U^{(k)} = b_j(x)\}$ is the associated active constraint set.

Problem (17) is an equality constrained QP. Let $\delta^{(k)} \in \mathbb{R}^n$ denote the optimizer of (17) and let $\lambda^{(k)} \in \mathbb{R}^{|\mathcal{A}|}$ be the corresponding vector of Lagrange multipliers. We note that $\delta^{(k)}$ is the best correction to U in the direction where the constraints $\mathcal{A}$ are active. Since (17) is just a special case of (10) it can be solved as explained

before. In particular, with the QR decomposition of $G_{\mathcal{A}}$ as in (14), and Z and Y defined by (15) we get

$$\delta^{(k)} = -Z(Z'HZ)^{-1}Z'(g(x) + HU), \quad (18)$$

$$\lambda^{(k)} = (Y' - Y'HZ(Z'HZ)^{-1}Z')(g(x) + HU). \quad (19)$$

Three cases are possible after solving (17):

- (1) $\delta^{(k)} = 0$. If all Lagrange multipliers are nonnegative then $U^{(k)}$ is the optimum for (7). Otherwise, find the constraint with the smallest Lagrange multiplier, remove it from $\mathcal{A}$, and iterate the procedure.
- (2) $\delta^{(k)} \neq 0$ and $U^{(k)} + \delta^{(k)}$ is feasible for (7). Set $U = U^{(k)} + \delta^{(k)}$, keep $\mathcal{A}$, and iterate the procedure.
- (3) $\delta^{(k)} \neq 0$ and $U^{(k)} + \delta^{(k)}$ is not feasible for (7). Then find the best feasible point $U = U^{(k)} + \alpha \delta^{(k)}$ with a line search in the direction of $\delta^{(k)}$. Add the newly activated constraints (by U) to $\mathcal{A}$ and iterate the procedure.

The whole procedure is summarized in Algorithm 1.

Algorithm 1 Active set method for QP (7)

Input: $H, g(x), G, b(x)$, initial feasible U ($GU \leq b(x)$)

Output: U^*

- 1: $k \leftarrow 1, \mathcal{A} \leftarrow \{i \in \mathcal{I} \mid G_i U = b_i(x)\}$
 - 2: $\mathcal{A}^{(k)} \leftarrow \mathcal{A}, U^{(k)} \leftarrow U$
 - 3: Compute $\delta^{(k)}$ by (18)
 - 4: **if** $\delta^{(k)} = 0$ **then**
 - 5: Compute $\lambda^{(k)}$ by (19)
 - 6: **if** $\lambda^{(k)} \geq 0$ **then**
 - 7: **return** $U^* \leftarrow U^{(k)}$
 - 8: **end if**
 - 9: $\mathcal{A} \leftarrow \mathcal{A} \setminus \mathcal{A}(\argmin_{i \in \mathcal{A}} \lambda_i^{(k)})$
 - 10: Compute $\delta^{(k)}$ by (18)
 - 11: **end if**
 - 12: Compute
 - 13: $\alpha = \min \left(1, \min_{j \in \mathcal{I} \setminus \mathcal{A}, G_j \delta^{(k)} > 0} \frac{b_j(x) - G_j U}{G_j \delta^{(k)}} \right)$
 - 14: $U \leftarrow U^{(k)} + \alpha \delta^{(k)}$
 - 15: **if** $\alpha < 1$ **then**
 - 16: $\mathcal{A} \leftarrow \mathcal{A} \cup \{j^*\}$, where j^* is the argmin of (20)
 - 17: **end if**
 - 18: $k \leftarrow k + 1, \text{goto } 2$
-

Alternative simple algorithm

The algorithm presented in this section is not typically used to solve QPs. It is introduced to better understand the issues involved with the explicit algorithms presented later in this paper. Compared to Algorithm 1, the algorithm proposed in this section does not require a feasible initial point and does not provide primal feasible solutions at intermediate steps.

At each iteration k , an active constraint set $\mathcal{A} = \mathcal{A}^{(k)}$ is known and the Eq. (11) is solved to obtain $U^{(k)} = U^*$ and $\lambda^{(k)} = \lambda^*$. The algorithm proceeds as follows:

- (1) If $U^{(k)}$ is not feasible for the QP (7), then find the constraint that is violated the most, included it into the active set $\mathcal{A}$ and iterate the procedure.
- (2) Find the smallest Lagrange multiplier. If it is nonnegative the optimum is found. Otherwise, remove the constraint it corresponds to from the active set $\mathcal{A}$ and iterate the procedure.

The two steps above are the main components of the Algorithm 2.

Remark 4.1. We remark that the storage demand, optimality certificate and active set selection for Algorithm 1 are identical to Algorithm 2. The only difference between Algorithms 1 and 2

Algorithm 2 Modified active set method for QP (7)**Input:** $H, g(x), G, b(x)$, initial $\mathcal{A}$ **Output:** U^*

```

1:  $k \leftarrow 1$ 
2:  $\mathcal{A}^{(k)} \leftarrow \mathcal{A}$ 
3: Compute  $U^{(k)}$  by (12a) and (16)
4:  $j^* \leftarrow \operatorname{argmin}_{j \in \mathcal{I} \setminus \mathcal{A}} b_j(x) - G_j U^{(k)}$ 
5: if  $G_{j^*} U^{(k)} > b_{j^*}$  then
6:    $\mathcal{A} \leftarrow \mathcal{A} \cup \{j^*\}$ 
7: else
8:   Compute  $\lambda^{(k)}$  by (12b) and (16)
9:   if  $\lambda^{(k)} \geq 0$  then
10:    return  $U^* \leftarrow U^{(k)}$ 
11:   end if
12:  $\mathcal{A} \leftarrow \mathcal{A} \setminus \mathcal{A}(\operatorname{argmin}_{i \in \mathcal{A}} \lambda_i^{(k)})$ 
13: end if
14:  $k \leftarrow k + 1$ , goto 2

```

is that the first guarantees primal feasibility of the intermediate solution at each step while the latter does not. This is obtained at the price of an initial feasible solution required to initialize Algorithm 1 and the additional steps in (20) required to compute a feasible solution.

4.2. Explicit solution: off-line and on-line computation

The basic idea behind the computation of the explicit solution to QP (7) for a set of parameters $x \in \mathcal{K} \subseteq \mathbb{R}^{n_x}$ can be described as follows. Consider an active set $\mathcal{A} = \mathcal{A}_i$ and the primal and dual solution U^*, λ^* in (12) with (16). We note that (12) actually shows that U^* and λ^* are affine functions of x since $b_{\mathcal{A}}(x)$ and $g(x)$ are affine functions of x , i.e.,

$$\begin{aligned} U^* &= (TB_{\mathcal{A}}^x - LF')x + Tb_{\mathcal{A}}^r =: F^{\mathcal{A}}x + c^{\mathcal{A}}, \\ \lambda^* &= (SB_{\mathcal{A}}^x - T'F')x + Sb_{\mathcal{A}}^r =: \Phi^{\mathcal{A}}x + \gamma^{\mathcal{A}}. \end{aligned} \quad (21)$$

Furthermore, the expressions $U^*(x)$ and $\lambda^*(x)$ remain constant for all x for which $\mathcal{A}_i$ is the active set at the optimum (note that in this section the lower index i in $\mathcal{A}_i$ simply denotes a counter, while in the previous section, the upper index (k) specifies variables at the k th step of the algorithm). Such region $\mathcal{CR}^{\mathcal{A}_i} \subseteq \mathbb{R}^{n_x}$ is called the “critical region” and is the set of all parameters x for which the constraints indexed by $\mathcal{A}_i$ are active at the optimizer of problem (7), i.e., $\mathcal{CR}^{\mathcal{A}_i} := \{x : \mathcal{A}^*(x) = \mathcal{A}_i\}$. The critical region $\mathcal{CR}^{\mathcal{A}_i}$ can be computed by imposing primal feasibility conditions (8d) on U^*

$$\mathcal{P}^{\mathcal{A}_i, p} := \{x \in \mathbb{R}^{n_x} \mid G_{\mathcal{I} \setminus \mathcal{A}_i} U^*(x) < b_{\mathcal{I} \setminus \mathcal{A}_i}(x)\} \quad (22)$$

and dual feasibility conditions (8c) on λ^*

$$\mathcal{P}^{\mathcal{A}_i, d} := \{x \in \mathbb{R}^{n_x} \mid \lambda^*(x) \geq 0\}. \quad (23)$$

In conclusion, the critical region $\mathcal{CR}^{\mathcal{A}_i}$ is the intersection of $\mathcal{P}^{\mathcal{A}_i, p}$ and $\mathcal{P}^{\mathcal{A}_i, d}$, i.e., $\mathcal{CR}^{\mathcal{A}_i} = \mathcal{P}^{\mathcal{A}_i, p} \cap \mathcal{P}^{\mathcal{A}_i, d}$. Obviously, the closure of $\mathcal{CR}^{\mathcal{A}_i}$ is a polyhedron in the x -space. Based on (21) we can rewrite the closure of the primal and dual polyhedron (22)–(23) as:

$$\begin{aligned} \mathcal{P}^{\mathcal{A}_i, p} &= \{x \in \mathbb{R}^{n_x} \mid G_{\mathcal{I} \setminus \mathcal{A}_i} F^{\mathcal{A}_i} x \leq b_{\mathcal{I} \setminus \mathcal{A}_i}(x) - G_{\mathcal{I} \setminus \mathcal{A}_i} c^{\mathcal{A}_i}\} \\ &= \{x \in \mathbb{R}^{n_x} \mid P_j^{\mathcal{A}_i} x \leq Q_j^{\mathcal{A}_i}, \text{ for } j \in \mathcal{I} \setminus \mathcal{A}_i\}, \end{aligned} \quad (24)$$

$$\begin{aligned} \mathcal{P}^{\mathcal{A}_i, d} &= \{x \in \mathbb{R}^{n_x} \mid -\Phi^{\mathcal{A}_i} x \leq \gamma^{\mathcal{A}_i}\} \\ &= \{x \in \mathbb{R}^{n_x} \mid P_j^{\mathcal{A}_i} x \leq Q_j^{\mathcal{A}_i}, \text{ for } j \in \mathcal{A}_i\}, \end{aligned} \quad (25)$$

thus obtaining

$$\mathcal{CR}^{\mathcal{A}_i} = \{x \in \mathbb{R}^{n_x} \mid P^{\mathcal{A}_i} x \leq Q^{\mathcal{A}_i}\}, \quad (26)$$

where $Q^{\mathcal{A}_i} \in \mathbb{R}^{|\mathcal{I}|} = \mathbb{R}^m$, $P^{\mathcal{A}_i} \in \mathbb{R}^{m \times n_x}$. Since the expression (26) may contain many redundant halfspaces, we may prefer to use the reduced description of $\mathcal{CR}^{\mathcal{A}_i}$

$$\mathcal{CR}^{\mathcal{A}_i} = \{x \in \mathbb{R}^{n_x} \mid P^i x \leq Q^i\}, \quad (27)$$

where $Q^i \in \mathbb{R}^{p_i}$, $P^i \in \mathbb{R}^{p_i \times n_x}$, and $p_i \leq m$ is the minimal number of halfspaces defining polyhedron $\mathcal{CR}^{\mathcal{A}_i}$.

An mp-QP algorithm determines the partition of $\mathcal{K}$ into critical regions $\mathcal{CR}^{\mathcal{A}_i}$, and finds the expression of the functions $U^*(\cdot)$ for each critical region. Let $N_{\mathcal{P}}$ denote the total number of critical regions. The explicit solution (21) of the QP (7) is

$$U^*(x) = F^{\mathcal{A}_i} x + c^{\mathcal{A}_i}, \quad \forall x \in \mathcal{CR}^{\mathcal{A}_i}, \quad i = 1, \dots, N_{\mathcal{P}}, \quad (28)$$

where $F^{\mathcal{A}_i} \in \mathbb{R}^{n \times n_x}$, $c^{\mathcal{A}_i} \in \mathbb{R}^n$, $\{\mathcal{CR}^{\mathcal{A}_i}\}_{i=1}^{N_{\mathcal{P}}}$ is a polyhedral partition of $\mathcal{K}^*$ (i.e., $\bigcup_i \mathcal{CR}^{\mathcal{A}_i} = \mathcal{K}^*$, and $\mathcal{CR}^{\mathcal{A}_i}$ and $\mathcal{CR}^{\mathcal{A}_j}$ have disjoint interiors $\forall i \neq j$), $i = 1, \dots, N_{\mathcal{P}}$. Note that in general $\bigcup_{i=1, \dots, N_{\mathcal{P}}} \mathcal{CR}^{\mathcal{A}_i} = \mathcal{K}^* \subseteq \mathcal{K}$ since for some $x \in \mathcal{K}$ the QP (7) could be infeasible.

The evaluation of explicit solutions in its simplest form would require: (i) the storage of the list of polyhedral regions and of the corresponding affine control laws, (ii) a sequential search through the list of polyhedra for the i th polyhedron that contains the current state in order to implement the i th control law. Since verifying if a point x belongs to a critical region means to verify primal and dual conditions, then the on-line search for the polyhedron containing x can be compared to the main steps of a QP solver. The simplest implementation consists of searching for the polyhedral region that contains the state x as in Algorithm 3.

Algorithm 3 Explicit solution to QP (7)**Input:** $x, P^i, Q^i, F^{\mathcal{A}_i}, c^{\mathcal{A}_i}, i = 1, \dots, N_{\mathcal{P}}$ **Output:** U^*

```

1: for  $i \leftarrow 1$  to  $N_{\mathcal{P}}$  do
2:   if  $P^i x \leq Q^i$  then
3:     return  $U^* \leftarrow F^{\mathcal{A}_i} x + c^{\mathcal{A}_i}$ 
4:   end if
5: end for

```

Remark 4.2. In MPC we need to store only the first n_u components of $F^{\mathcal{A}_i}$ and $c^{\mathcal{A}_i}$, see (6). Denote by $N_{\mathcal{H}}$ the total number of halfspaces defining the feasible polyhedral partition: $N_{\mathcal{H}} := \sum_{i=1}^{N_{\mathcal{P}}} p_i$. It is easy to see that Algorithm 3 requires $(n_x + 1)N_{\mathcal{H}}$ real numbers to store all polyhedra $\mathcal{CR}^{\mathcal{A}_i}$ and $n_u N_{\mathcal{P}}(n_x + 1)$ real numbers to store the control laws, for a total of

$$\# \text{reals} = (n_x + 1)N_{\mathcal{H}} + n_u N_{\mathcal{P}}(n_x + 1). \quad (29)$$

Furthermore, in the worst case (when the state is contained in the last region of the list) Algorithm 3 will give a solution after $n_x N_{\mathcal{H}}$ multiplications, $(n_x - 1)N_{\mathcal{H}}$ sums and $N_{\mathcal{H}}$ comparisons. Hence,

$$\# \text{operations} \leq 2n_x N_{\mathcal{H}}. \quad (30)$$

Algorithm 3 differs from Algorithm 2 in the way the next active set (i.e. region) is chosen. One can easily modify Algorithm 3 to use the same active strategy of Algorithm 2 as follows. In the i th critical region $\mathcal{CR}^{\mathcal{A}_i}$ we separate the constraints deriving from primal feasibility $P_{\mathcal{I} \setminus \mathcal{A}_i}^{\mathcal{A}_i} x \leq Q_{\mathcal{I} \setminus \mathcal{A}_i}^{\mathcal{A}_i}$ and the constraints deriving from dual feasibility $P_{\mathcal{A}_i}^{\mathcal{A}_i} x \leq Q_{\mathcal{A}_i}^{\mathcal{A}_i}$. This idea is used in Algorithm 4, which can be interpreted as the “off-line version” of Algorithm 2. In Algorithm 4 we use the function $\text{idx}_{\mathcal{A}}(\mathcal{B})$ to describe the following operation: given the set $\mathcal{B} \in \mathbb{A}$, where $\mathbb{A} := \{\mathcal{A}_1, \dots, \mathcal{A}_{N_{\mathcal{P}}}\}$, extract the index i such that $\mathcal{A}_i = \mathcal{B}$.

The “off-line version” of Algorithm 1 can be found in Borrelli, Baotic, Pekar, and Stewart (2010). Note that in such an algorithm the parameter space will comprise the system current state x and the (feasible) solution at the previous time step $U^{(k)}$.

Remark 4.3. If redundant constraints have been removed from primal and dual polyhedra, then Steps 3, 5, 7, 9 of Algorithm 4 need to be modified accordingly.

Algorithm 4 Off-line version of Algorithm 2**Input:** $x, P^{\mathcal{A}_i}, Q^{\mathcal{A}_i}, F^{\mathcal{A}_i}, c^{\mathcal{A}_i}, i = 1, \dots, N_{\mathcal{P}}$, initial i **Output:** U^*

```

1: for  $k \leftarrow 1$  to  $N_{\mathcal{P}}$  do
2:    $\mathcal{A}^{(k)} \leftarrow \mathcal{A}_i$ 
3:    $j^* \leftarrow \operatorname{argmin}_{j \in \mathcal{I} \setminus \mathcal{A}_i} Q_j^{\mathcal{A}_i} - P_j^{\mathcal{A}_i} x$ 
4:   if  $P_{j^*}^{\mathcal{A}_i} x > Q_{j^*}^{\mathcal{A}_i}$  then
5:      $i \leftarrow \operatorname{idx}_{\mathcal{A}}(\mathcal{A}_i \cup \{j^*\})$ 
6:   else
7:      $j^* \leftarrow \operatorname{argmin}_{j \in \mathcal{A}_i} Q_j^{\mathcal{A}_i} - P_j^{\mathcal{A}_i} x$ 
8:     if  $P_{j^*}^{\mathcal{A}_i} x > Q_{j^*}^{\mathcal{A}_i}$  then
9:        $i \leftarrow \operatorname{idx}_{\mathcal{A}}(\mathcal{A}_i \setminus \{j^*\})$ 
10:    else
11:      return  $U^* \leftarrow F^{\mathcal{A}_i} x + c^{\mathcal{A}_i}$ 
12:    end if
13:  end if
14: end for

```

5. Comparison between on-line and explicit algorithms

By comparing Algorithms 2 and 4 we can draw the following conclusions:

- (i) The active set strategy and the certificate of optimality are the same. In particular, Step 8 in Algorithm 2 corresponds to Step 7 in Algorithm 4. The difference is clear from Eq. (25). In the explicit Algorithm 4 the matrices $P_{\mathcal{A}_i}^{\mathcal{A}_i}$ and $Q_{\mathcal{A}_i}^{\mathcal{A}_i}$ are precomputed and stored. In the on-line QP Algorithm 2, $P_{\mathcal{A}_i}^{\mathcal{A}_i}$ and $Q_{\mathcal{A}_i}^{\mathcal{A}_i}$ are computed on-line from (12b) and (16). Similarly, Step 4 in Algorithm 2 requires the computation of $b_{\mathcal{I} \setminus \mathcal{A}}(x) - G_{\mathcal{I} \setminus \mathcal{A}} U^{(k)}$ which corresponds to Step 3 in Algorithm 4. The difference is clear from Eq. (24). In the explicit Algorithm 4 the matrices $P_{\mathcal{I} \setminus \mathcal{A}_i}^{\mathcal{A}_i}$ and $Q_{\mathcal{I} \setminus \mathcal{A}_i}^{\mathcal{A}_i}$ are precomputed and stored. In the on-line QP Algorithm 2, $P_{\mathcal{I} \setminus \mathcal{A}_i}^{\mathcal{A}_i}$ and $Q_{\mathcal{I} \setminus \mathcal{A}_i}^{\mathcal{A}_i}$ are computed on-line from (12a) and (16). Although Eqs. (12) and (16) can be efficiently computed, on-line QR decomposition and several matrix multiplications are required at each time step. This is the cornerstone of the computational saving that can be achieved by explicit solvers compared to on-line active set QPs.
- (ii) Any modification which helps computing the L , T , and S in (16) will improve performance of Algorithm 2. However, a computational gap between Algorithms 2 and 4 will always exist unless those matrices are pre-computed as in Algorithm 4. Any modification of the active set strategy or the optimality certificate can be applied to the explicit solution as well and therefore it will not affect the comparison.
- (iii) Algorithms 2 and 4 can be properly compared only if they are initialized with the same set $\mathcal{A}$. Note that there might be active constraint sets for which it is possible to initialize Algorithm 2 and not Algorithm 4. In fact, Algorithm 4 stores only full-dimensional critical regions for which the corresponding set $\mathcal{A}^{(k)}$ is active at $U^*(x)$ for some x , i.e., $\mathcal{A}^*(x)$ defined in (9).

The following proposition shows that if Algorithms 2 and 4 are initialized with the same set $\mathcal{A}$ then they explore the same active sets.

Proposition 1. Assume that all the critical regions (27) of the QP (7) are full-dimensional polyhedra. Let x be a feasible state and consider Algorithm 4. If, at any time step k , $\mathcal{A}^{(k)}$ corresponds to a full-dimensional critical region stored off-line by Algorithm 4, then $\mathcal{A}^{(k+1)} = \mathcal{A}_i$, with $i = \operatorname{idx}_{\mathcal{A}}(\mathcal{A}^{(k)} \cup \{j^*\})$ and j^* computed as in Step 3 of Algorithm 4 (or $i = \operatorname{idx}_{\mathcal{A}}(\mathcal{A}^{(k)} \setminus \{j^*\})$ and j^* computed as in Step 7 of Algorithm 4), corresponds to a full-dimensional critical region that has been stored off-line by Algorithm 4.

At the generic step k of Algorithm 4, assume that constraint j of the current critical region $\mathcal{C}\mathcal{R}^{\mathcal{A}^{(k)}}$ (say $a_j x \leq b_j$) is violated. Let $\mathcal{A}^{(k+1)} = \mathcal{A}_i$, with $i = \operatorname{idx}_{\mathcal{A}}(\mathcal{A}^{(k)} \cup \{j\})$ ($i = \operatorname{idx}_{\mathcal{A}}(\mathcal{A}^{(k)} \setminus \{j\})$), be the new set of active constraints if j is a primal (dual) constraint. Then, three cases can occur: (1) $\mathcal{C}\mathcal{R}^{\mathcal{A}^{(k+1)}}$ has been stored off-line by Algorithm 4, (2) $\mathcal{C}\mathcal{R}^{\mathcal{A}^{(k+1)}}$ is not a full-dimensional critical region and (3) $\mathcal{C}\mathcal{R}^{\mathcal{A}^{(k+1)}}$ is empty. Case (2) is excluded by assumption. Case (3) can also be excluded by contradiction. In fact, assume there is no critical region in the half-space $a_j x > b_j$. Then $a_j x \leq b_j$ is a facet of the region of feasible states $\mathcal{K}$ and therefore (since it is violated) x is not feasible, which contradicts the assumption. Therefore, the only admissible option is case (1) which proves the proposition. $\square$

Note that the presence of critical regions which are not full-dimensional is linked to degeneracies of the QP (7), see Spjøtvold, Kerrigan, Jones, Tøndel, and Johansen (2006) and Tøndel, Johansen, and Bemporad (2003) for more details.

- (iv) When computing the critical regions (27) redundant constraints are removed. While, in general, this can improve the efficiency of Algorithm 4 versus Algorithm 2, by definition these constraints will not play any role in selecting the next set of active constraints.
- (v) From the above observation it is clear that Algorithm 4 requires less operations at each iteration than Algorithm 2. This is obtained at the price of increased memory requirement. In fact, in Algorithm 4 the polyhedral partition and the gains have to be stored which, in general, largely surpass the memory required for Algorithm 2 (simply the matrices of the QP (7)).

Comparing Algorithms 1 and 4 might be of interest since they are often the preferred choices of on-line and off-line methods, respectively. The following observations should be considered. (1) If Algorithm 1 is initialized with a primal feasible solution then it maintains primal feasibility. Algorithm 4 does not maintain primal feasibility. Note that the computation of an initial feasible solution might be a time-consuming task. By avoiding this step, Algorithm 4 improves on the average computational time. We remark that an efficient method for a warm-start of an active set strategy was presented in Zeilinger, Jones, and Morari (2008). (2) In Algorithms 1 and 4, the active set strategy and the certificate of optimality are the same. In particular, Step 5 in Algorithm 1 corresponds to Step 7 in Algorithm 4. Similarly, Step 12 in Algorithm 1 requires the computation of $b_j(x) - G_j U^{(k)}$ which corresponds to Step 3 in Algorithm 4. The additional part of Step 12 in Algorithm 1 consists in dividing $b_j(x) - G_j U^{(k)}$ by $G_j \delta^{(k)}$ and computing α . This is not required in the explicit solution because primal feasibility is not maintained in the explicit QP and thus the smallest step $\delta^{(k)}$ guaranteeing primal feasibility does not need to be computed. (3) Algorithms 1 and 4 can be properly compared only if they have the same initial set $\mathcal{A}$. The number of such $\mathcal{A}$ might be small.

Remark 5.1. Obtaining $\mathcal{A}^{(k+1)}$ from $\mathcal{A}^{(k)}$ (i.e., using the function $\operatorname{idx}_{\mathcal{A}}(\cdot)$) is immediate in Algorithm 1 since it corresponds to a simple selection of different matrix rows. In Algorithm 4, if the list of neighboring regions is available, then $\operatorname{idx}_{\mathcal{A}}(\mathcal{A}^{(k)} \cup \{j^*\})$ (or $\operatorname{idx}_{\mathcal{A}}(\mathcal{A}^{(k)} \setminus \{j^*\})$) is not time consuming since it corresponds to identifying the neighboring region of a given facet of $\mathcal{C}\mathcal{R}^{\mathcal{A}^{(k)}}$. However, the construction of neighboring region list can be a numerically sensitive issue for mp-QP solvers especially in the case of degeneracies. If the list of neighboring regions is not available then, the operation $\operatorname{idx}_{\mathcal{A}}(\cdot)$ requires a search through a list of active constraints associated to all the stored region and it might be time consuming.

Remark 5.2. The comparison of Algorithms 3 and 2 is similar to the comparison between Algorithms 2 and 4 with two main differences. Algorithm 3 corresponds to an active set QP where the next active set is chosen according to a preassigned sequence. Although, on average it might perform worse than Algorithm 4, it does not require the computation of all dual variables as in Step 3 of Algorithm 4 and of all primal feasibility conditions as in Step 7 of Algorithm 4. As soon as one is violated, the algorithm moves to the next region in the list. This does reduce the computational time of each step. Secondly, Algorithm 3 works well even in presence of non-full-dimensional critical regions (in fact it does not require the list of neighboring regions). For these reasons, Algorithm 3 is very simple and practical even if it might perform very poorly on average.

6. Semi-explicit algorithm

By using the results of the previous section we can define alternative algorithms which trade off memory and computational time in a different way compared to both active set QPs and explicit solvers. Next we present an approach that, for a class of MPC problems, reduces both required memory and worst case computational time compared to an explicit solver.

Without loss of generality we assume that $g(x) = 0$ (for QP deriving from MPC problems this can always be obtained by a simple coordinate transformation (Bemporad et al., 2002)). Consider Eqs. (12) and (13), and rewrite $U^*(x)$ and $\lambda^*(x)$ as

$$\begin{aligned} U^* &= H^{-1}G_{\mathcal{A}}'(G_{\mathcal{A}}H^{-1}G_{\mathcal{A}}')^{-1}b_{\mathcal{A}}(x) =: T^{\mathcal{A}}b_{\mathcal{A}}(x) \\ \lambda^* &= -(G_{\mathcal{A}}H^{-1}G_{\mathcal{A}}')^{-1}b_{\mathcal{A}}(x) =: S^{\mathcal{A}}b_{\mathcal{A}}(x). \end{aligned} \quad (31)$$

The main idea is to compute and store the matrices $T^{\mathcal{A}} \in \mathbb{R}^{n \times |\mathcal{A}|}$ and $S^{\mathcal{A}} \in \mathbb{R}^{|\mathcal{A}| \times |\mathcal{A}|}$ off-line for all optimal sets of active constraints $\mathcal{A}$. Then, the on-line steps are: (1) compute $b(x) = b^r + B^x x$, verify duality conditions by computing λ^* from (31), compute the optimizer candidate U^* by using (31) and verify primal feasibility conditions $GU^* \leq b(x)$. We report implementation of this idea in Algorithm 5.

Algorithm 5 Semi-explicit solution to QP (7)

Input: $x, b^r, B^x, T^{\mathcal{A}_i}, S^{\mathcal{A}_i}, i = 1, \dots, N_{\mathcal{P}}$, initial $\mathcal{A} \neq \emptyset$

Output: U^*

```

1:  $k \leftarrow 1, \mathcal{A}^{(k)} \leftarrow \mathcal{A}$ 
2:  $b \leftarrow b^r + B^x x$ 
3: if  $b \geq 0$  then
4:   return  $U^* \leftarrow 0$ 
5: end if
6:  $i^* \leftarrow \operatorname{argmin}_i S_{i,i}^{\mathcal{A}^{(k)}} b_{\mathcal{A}^{(k)}}$ 
7: if  $S_{i^*,i^*}^{\mathcal{A}^{(k)}} b_{\mathcal{A}^{(k)}} < 0$  then
8:    $\mathcal{A}^{(k+1)} \leftarrow \mathcal{A}^{(k)} \setminus \mathcal{A}^{(k)}(i^*)$ 
9: else
10:   $z \leftarrow T^{\mathcal{A}^{(k)}} b_{\mathcal{A}^{(k)}}$ 
11:   $j^* \leftarrow \operatorname{argmin}_j b_j - G_j z(x)$ 
12:  if  $G_{j^*} z > b_{j^*}$  then
13:     $\mathcal{A}^{(k+1)} \leftarrow \mathcal{A}^{(k)} \cup j^*$ 
14:  else
15:    return  $U^* \leftarrow z$ 
16:  end if
17: end if
18:  $k \leftarrow k + 1, \text{goto } 6$ 
```

It is clear that since we do not store the primal feasible polyhedron, Algorithm 5 might save memory compared to an explicit algorithm. To be precise, Algorithm 5 requires mn real

numbers to store matrix G , m to store b^r , mn_x to store B^x , $n \sum_{k=1}^{N_{\mathcal{P}}} |\mathcal{A}^{(k)}|$ to store all $T^{\mathcal{A}^{(k)}}$ and $\sum_{k=1}^{N_{\mathcal{P}}} |\mathcal{A}^{(k)}|^2$ for all $S^{\mathcal{A}^{(k)}}$. In total we get a total storage demand of

$$\# \text{reals} = m(n + n_x + 1) + \sum_{k=1}^{N_{\mathcal{P}}} (n|\mathcal{A}^{(k)}| + |\mathcal{A}^{(k)}|^2). \quad (32)$$

The worst case number of operations are required when (i) the state is contained in the last region and (ii) for every region only the last constraint in the list is not satisfied. The total worst case number of operation (see Borrelli et al., 2010 for details) is

$$m(n_x + 1) + N_{\mathcal{P}}n(2m - 1) + 2 \sum_{k=1}^{N_{\mathcal{P}}} |\mathcal{A}^{(k)}|^2. \quad (33)$$

In conclusion we can combine the results in (30) with (33) and state that Algorithm 5 is less computationally demanding than the explicit Algorithm 3 if

$$2n_x N_{\mathcal{H}} > m(n_x + 1) + N_{\mathcal{P}}n(2m - 1) + 2 \sum_{k=1}^{N_{\mathcal{P}}} |\mathcal{A}^{(k)}|^2. \quad (34)$$

Also, combining (29) with (32) we obtain that Algorithm 5 saves memory compared to the explicit Algorithm 3 if

$$(n_x + 1)N_{\mathcal{H}} + n_u N_{\mathcal{P}}(n_x + 1) > m(n + n_x + 1) + \sum_{k=1}^{N_{\mathcal{P}}} (n|\mathcal{A}^{(k)}| + |\mathcal{A}^{(k)}|^2). \quad (35)$$

If in every region there are no redundant constraints (this is often true when the number of parameters is large), then $N_{\mathcal{H}} = mN_{\mathcal{P}}$ and inequality (34) for large $N_{\mathcal{P}}$ can be approximated with a simple expression

$$n_x m > n(2m - 1) + \frac{2}{N_{\mathcal{P}}} \sum_{k=1}^{N_{\mathcal{P}}} |\mathcal{A}^{(k)}|^2. \quad (36)$$

Note that (36) links the efficiency of the proposed algorithm to the number of parameters n_x , the number of optimization variables n , the number of constraints m and the average number of squared active constraints $\frac{1}{N_{\mathcal{P}}} \sum_{k=1}^{N_{\mathcal{P}}} |\mathcal{A}^{(k)}|^2$. Examples showing the effectiveness of the proposed approach, conclusions and future research topic can be found in Borrelli et al. (2010).

References

- Baotić, M., Borrelli, F., Bemporad, A., & Morari, M. (2008). Efficient on-line computation of constrained optimal control. *SIAM Journal on Control and Optimization*, 47, 2470–2489.
- Bemporad, A., Morari, M., Dua, V., & Pistikopoulos, E. N. (2002). The explicit linear quadratic regulator for constrained systems. *Automatica*, 38, 3–20.
- Borrelli, F., Baotić, M., Pekar, J., & Stewart, G. (2010). On the complexity of explicit MPC laws. *UC Berkeley Technical report*. Online at: www.mpc.berkeley.edu.
- Ferreau, H. J., Bock, H. G., & Diehl, M. (2008). An online active set strategy to overcome the limitations of explicit MPC. *International Journal of Robust and Nonlinear Control*, 18, 816–830.
- Fletcher, R. (1987). *Practical methods of optimization* (second ed.). Wiley-Interscience Publication.
- Jones, C., Grieder, P., & Raković, S. (2005). A logarithmic-time solution to the point location problem for closed-form linear MPC. In *IFAC world congress*. July. Prague, Czech Republic.
- Mayne, D. Q., Rawlings, J. B., Rao, C. V., & Scokaert, P. O. M. (2000). Constrained model predictive control: stability and optimality. *Automatica*, 36, 789–814.
- Milman, R., & Davison, E. J. (2008). A fast MPC algorithm using nonfeasible active set methods. *Journal of Optimization Theory and Applications*. Published on-line.
- Santos, L. O., Afonso, P., Castro, J., Oliveira, N., & Biegler, L. T. (2001). On-line implementation of nonlinear mpc: an experimental case study. *Control Engineering Practice*, 9(8), 847–857.
- Spjøtvold, J., Kerrigan, E. C., Jones, C. N., Tøndel, P., & Johansen, T. A. (2006). On the facet-to-facet property of solutions to convex parametric quadratic programs. *Automatica*, 42, 2209–2214.
- Tøndel, P., Johansen, T. A., & Bemporad, A. (2003). An algorithm for multiparametric quadratic programming and explicit MPC solutions. *Automatica*, 39, 489–497.
- Wang, Y., & Boyd, S. (2008). Fast model predictive control using online optimization. In *Proceedings IFAC world congress* (pp. 6974–6997). July. Seoul.

Zeilinger, M., Jones, C. N., & Morari, M. (2008). Real-time suboptimal model predictive control using a combination of explicit MPC and online optimization. In *Conference on decision and control, CDC 2008*. December. Cancun, Mexico.



he was appointed the chair of the IEEE technical committee on automotive control. His research interests include constrained optimal control, model predictive control and its application to advanced automotive control and energy efficient building operation.

Francesco Borrelli received the 'Laurea' degree in computer science engineering in 1998 from the University of Naples 'Federico II', Italy. In 2002 he received the Ph.D. from the Automatic Control Laboratory at ETH-Zurich, Switzerland. He is currently an Assistant Professor at the Department of Mechanical Engineering of the University of California at Berkeley, USA. He is the author of more than fifty publications in the field of predictive control. He is author of the book *Constrained Optimal Control of Linear and Hybrid Systems* published by Springer Verlag and the winner of the 2009 NSF CAREER award. In 2008



Zagreb, Croatia. His research interests include mathematical programming, hybrid systems, optimal control and model predictive control.

Mato Baotić received the B.Sc. and M.Sc. degrees, both in Electrical Engineering, from the Faculty of Electrical Engineering and Computing (FER Zagreb), University of Zagreb, Croatia, in 1997 and 2000, respectively. As a recipient of the ESKAS scholarship of the Swiss Government he was a visiting researcher at the Automatic Control Lab, Swiss Federal Institute of Technology (ETH) Zurich, Switzerland, during the academic year 2000/2001. In 2005 he received the Ph.D. from the ETH Zurich, Switzerland. Currently he is Assistant Professor at the Department of Control and Computer Engineering, FER



Jaroslav Pekar is a Research Engineer in Honeywell Prague Laboratory (Honeywell ACS AT Laboratories). He has been with Honeywell since 2005. Jaroslav is working on R&D projects focused on optimal and advanced control design for wide range of applications, e.g. fast sampled dynamical systems, power generation, industrial energy, etc. He holds Ph.D. degree in Control Engineering from the Faculty of Electrical Engineering at the Czech Technical University in Prague.



Greg Stewart received the B.Sc. degree in Physics in 1994 and the M.Sc. degree in Applied Mathematics in 1996 from Dalhousie University in Halifax, and the Ph.D. degree from the Department of Electrical and Computer Engineering at the University of British Columbia in Vancouver, Canada in 2000.

He currently holds the position of Fellow in Honeywell Automation and Control Solutions and has held an Adjunct Professor appointment in the Department of Electrical and Computer Engineering at the University of British Columbia since 2000. He was appointed Associate Editor for the IEEE Transactions on Control Systems Technology in 2005.

His research interests are in the development and use of theory for industrial implementation of advanced control strategies. He holds 16 patents, has published more than 50 technical publications, and his designs reside on over 200 industrial installations.

Dr. Stewart has received the IEEE Control Systems Technology Award, the IEEE Transactions on Control Systems Technology Outstanding Paper Award, an NSERC University–Industry Synergy Award for Innovation, and four Honeywell Technical Achievement awards.